Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 1

Introduction to Operating Systems

What an operating system does, how a computer system is organised, and the ideas the rest of the course builds on.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Describe the general organisation of a computer system and the role of interrupts.
2. Describe the components of a modern multiprocessor computer system.
3. Explain the transition from user mode to kernel mode.
4. Discuss how operating systems are used in different computing environments.
5. Give examples of free and open-source operating systems.

THIS WEEK

Topic 1

An overview. Every topic here returns later in the course in more depth.

DISCUSSION

What happens when you double-click an icon?

Something finds the program on disk, gives it memory, lets it use the processor, and draws its window. That something is our subject.

Why learn operating systems?

Every program you will ever write runs *on top of one*.

Your code depends on it

Files, memory, the network, threads and the clock are all services of the operating system. Your program only asks for them.

It decides how fast you run

Scheduling, caching, paging and I/O explain why the same code is fast on one day and slow on the next.

It explains the hard bugs

Race conditions, deadlocks, memory leaks and crashes only make sense once you know what happens below your code.

Where you will meet it in your work



Backend and web

A server handles many requests at once with processes and threads.



Cloud and DevOps

Containers and virtual machines are operating-system features.



Mobile and embedded

Little memory, a battery and strict time limits.



Security

Users, permissions, isolation and privilege are enforced by the kernel.



Data and AI

Large data sets and models depend on memory, storage and many processors.



Debugging

Reading a crash, following system calls, finding what holds a lock.

Ideas you will use again

The operating system is the oldest large piece of software most of us use. Its solutions were copied by databases, browsers, game engines and distributed systems.

01

Abstraction

Hide a complicated device behind a simple interface, such as the file.

02

Concurrency

Let many things happen at once without corrupting shared data.

03

Scheduling

Share a limited resource fairly among many who want it.

04

Caching

Keep what is used often close at hand, and decide what to throw away.

About this course

40%

Final exam

30%

Labs and project

Lab assignments and the programming project.

20%

Midterm exam

10%

Quizzes

With class participation.

CLO1: analyze

Operating-system architectures, process management, memory management and resource allocation.

CLO2: implement

System-level programs, Linux administration tasks, shell scripts and synchronization mechanisms.

CLO3: organize

Technical solutions and documentation, with professional accountability.

Source: course specification

The semester

Weeks	Lectures	Labs
1 to 4	Introduction, operating system structure, processes, threads	Linux navigation, files and directories, redirection and pipelines, ownership and permissions
5 to 7	CPU scheduling, critical section and synchronization, review	Process management in Linux, shell scripting
8	Midterm examination	
9 to 11	Synchronization I and II, deadlocks	Advanced scripting, scheduling tasks with cron
12 to 14	Memory management, virtual memory, review	Programming project
16	Final examination and project submission	

Source: course specification

Course policy and what you need

Below 30 out of 100

An overall score below 30% means a re-sit of the exam.

Absence

Absence without permission for more than 50% of the course hours is an automatic fail.

Project updates

More than three weeks with no weekly project update means zero for the poster presentation.

On your own computer: a virtual machine (VMware Workstation Pro), Ubuntu 20.04 or later, the GCC compiler, and a text editor such as vim, nano or VS Code.

Source: course specification

How this course runs

Lectures, class activities and labs, all kept in one repository.

Everything is in one repository

Lectures

The chapter slides, and one note-taking template for each week's topic.

Class activities

Hands-on programming tasks that go with the lecture topics.

Labs

Eleven lab instructions, the environment setup guide and the report template.

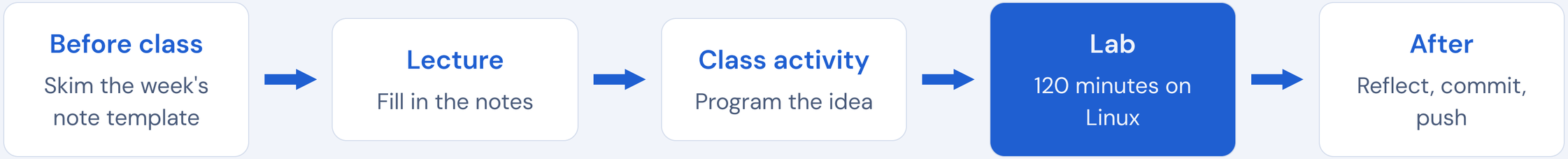
Visualizations

Step-through animations of semaphores, deadlocks, paging and page replacement.

github.com/RathpiseyAlpha/ITC-I4GIC-OS. Start with the README, then the course outline.

Source: course repository

How a week works



Before

Open the note template for the coming week and read its key questions.

During

Write the definitions, the diagrams and the key points as the lecture goes.

After

Finish the reflection, add your own examples, and write a summary of three to five sentences.

Source: course repository

Class activities

Each activity gives you practice with the concept from the lecture. Most are programming tasks; one is reasoning on paper with the visualizations.

01

Read everything first

Read the whole instruction, top to bottom, before you write any code.

02

Set up

A Linux virtual machine, WSL or native Linux.

03

Required, then bonus

Finish the required tasks before you try the optional ones.

04

Submit

Your code and a README with your name, student ID, screenshots and answers.

Source: course repository

The class activities so far

	Lecture topic	What you do
1	Operating-system structures and interfaces	System calls with POSIX, in C
2	Processes and threads	Processes and inter-process communication
3	Processes and threads	Socket communication and multithreading
4	Critical sections and synchronization	A shared file, a C++ mutex and Java synchronization
5	Semaphores and deadlocks	Semaphores for a particle buffer, and ordering output
6	Deadlocks	A bank-transaction deadlock, then its prevention
7	Deadlocks	Resource-allocation graphs and the banker's algorithm, no code
8	Memory management and virtual memory	Paging and page replacement: trace by hand, then simulate

Source: course repository

What we do in the labs

You work on a real Linux system, predict what it will do, and then check.

Where the labs run

PRIMARY

The shared Ubuntu server

The main environment. Each student has an individual account. The tools are prepared before class and you do not need sudo.

FALLBACK

Local Linux or WSL

A fallback for the tasks that need no special privileges. You clone the repository and define the lab helper yourself.

FOR ADMIN WORK

A disposable virtual machine

For boot and system-administration practice, with a snapshot. This work is never done on the shared server.

Use only your own account, your own files and your own processes.

Source: course repository

Inside a 120-minute lab

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	A guided example: commands and what you observe
25 to 35	Your written prediction, with an optional five-minute comparison with a classmate
35 to 70	The numbered investigation tasks, done individually
70 to 85	A normal test and an edge test, with feedback from the instructor
85 to 100	The individual checkpoint on a changed case
100 to 110	Your correction and explanation of the concept
110 to 120	Cleanup and submission

Source: course repository

How you work in a lab

Predict
On paper, before you run it

→

Investigate
Follow the numbered tasks

→

Test
A normal and an edge case

→

Checkpoint
A changed case, alone

→

Explain
Correct your prediction

Individual work

Each student submits their own solution. A short exchange with a classmate is optional.

AI tools

Allowed for the investigation and the tests. Not for the first prediction, and not for the checkpoint.

Marks

Each lab has a 10-point rubric tied to its objectives. The checkpoint is collected separately.

Source: course repository

What you learn in the labs

Lab	Core topic
1	Inspecting the system and your own processes
2	Navigation and placing files
3	Links and user-local libraries
4	Pipelines, redirection and processes
5	Processes, threads and joins
6	Permissions and access decisions

Lab	Core topic
7	Bash arguments and safe paths
8	A race on shared stock, and the critical section
9	Diagnosing a deadlock and recovering
10	Backup retention and scheduling with cron
11	Disk images and file-system evidence (optional bonus)

Source: course repository

What you hand in

```
os-se-<YourStudentID>/
  os-lab-<YourStudentID>/
    lab1/ README.md, sources, evidence/
    lab2/
  os-class-activities-<YourStudentID>/
    activity1/ README.md, screenshots/, task1/
    activity2/
```

A lab report holds

- The prediction you wrote before running anything.
- The script, or at most ten decisive command lines.
- Two test records: a normal case and an edge case.
- Your correction and a short explanation of the mechanism.

Keep the original prediction even when it was wrong. Commit and push regularly.

Source: course repository

PART 1

What operating systems do

A program that sits between the user's programs and the hardware.

What is an operating system?

A program that acts as an *intermediary* between the user of a computer and the computer hardware.

Run programs

It executes user programs and makes solving the user's problems easier.

Convenience

It makes the computer system convenient to use.

Efficiency

It uses the computer hardware in an efficient manner.

Four components of a computer system

Users

people, machines, other computers

Application programs

compilers, browsers, games, databases

Operating system

controls and coordinates the hardware

Computer hardware

CPU, memory, I/O devices

Each layer uses the one below it.

The hardware provides the basic computing resources.

Application programs define how those resources are used to solve problems.

The operating system coordinates the use of the hardware among the applications and the users.

What the operating system does depends on the view



The user's view

Ease of use and good performance. A laptop user does not care how resources are shared. A phone user cares about battery life. An embedded device may have no user interface at all.



The system's view

A resource allocator: it decides who gets the CPU, memory and devices, fairly and efficiently. And a control program: it controls how programs run, to prevent errors and improper use.

Defining the operating system

ALWAYS RUNNING

Kernel

The one program running at all times on the computer. This is the usual definition of the operating system.

COMES WITH IT

System programs

Programs that ship with the operating system but are not part of the kernel.

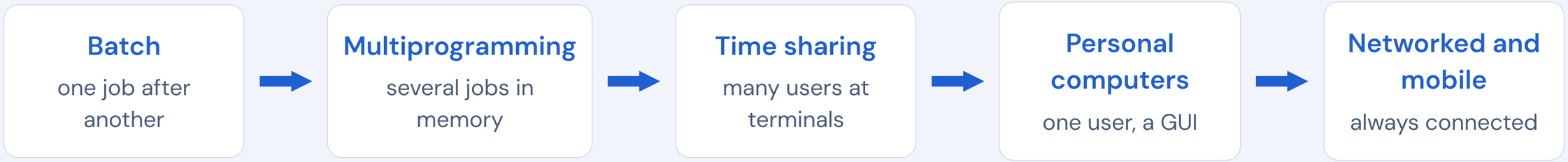
EVERYTHING ELSE

Application programs

Every program that is not needed to operate the system.

There is no single accepted definition. Today's systems also include middleware: frameworks that give developers extra services, such as databases and graphics.

How operating systems evolved



Each step answered a problem of the one before: idle CPUs, users who waited hours for results, machines too costly to own, and computers that could not reach each other.

Types of operating systems

Batch

Jobs are collected and run in sequence, with no interaction.

Time-sharing

The CPU is shared among many interactive users.

Distributed

Several machines work together and appear as one system.

Network

Machines share files and printers, and each remains visible.

Real-time

Results must arrive within fixed time limits.

Mobile and embedded

Limited memory and power, often for a single purpose.

Services and interfaces

Services

Run programs, do I/O, manage files, let processes communicate, detect errors, allocate resources and protect them.

For users

A command line, a graphical interface or a touch screen.

For programs

System calls: the functions through which a program asks the kernel for a service.

Week 2 looks at each of these in detail.

PART 2

Computer-system organisation

CPUs and device controllers share a bus and compete for memory cycles.

The von Neumann architecture

A program is *data*: the instructions and the values they work on are kept in the same memory.

Stored program

To run a different program you load different contents into memory. The hardware does not change.

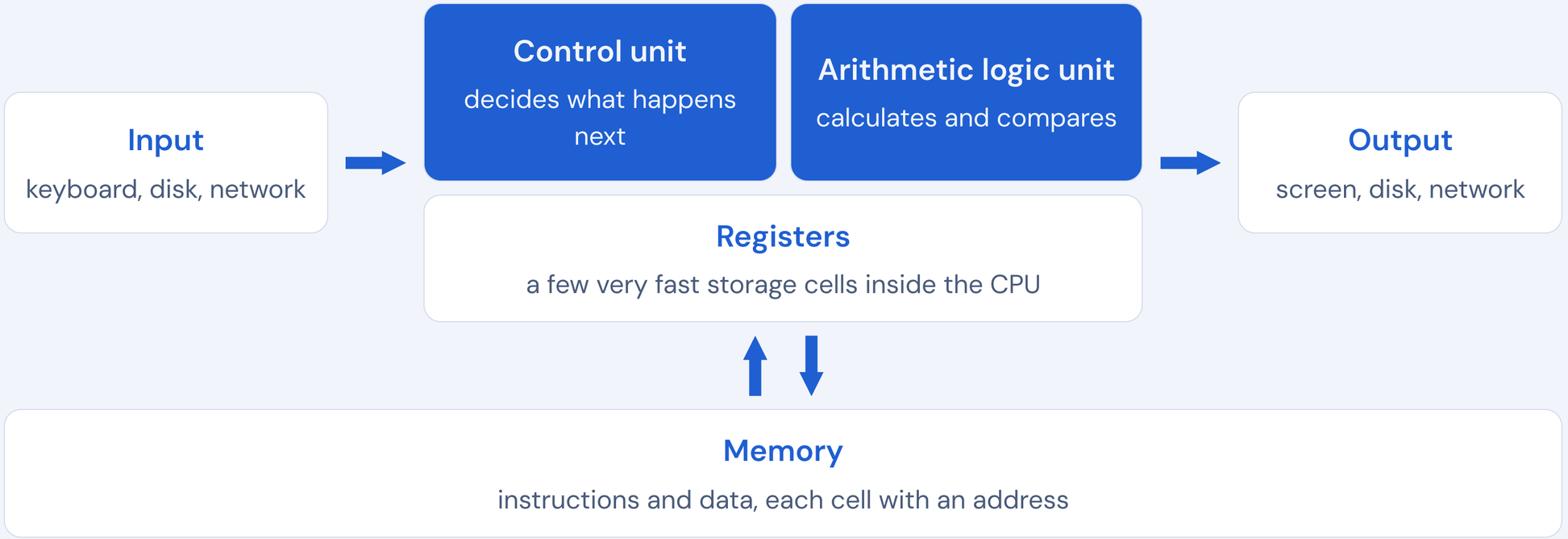
One step at a time

A processor takes one instruction from memory, carries it out, and goes on to the next.

Still the model

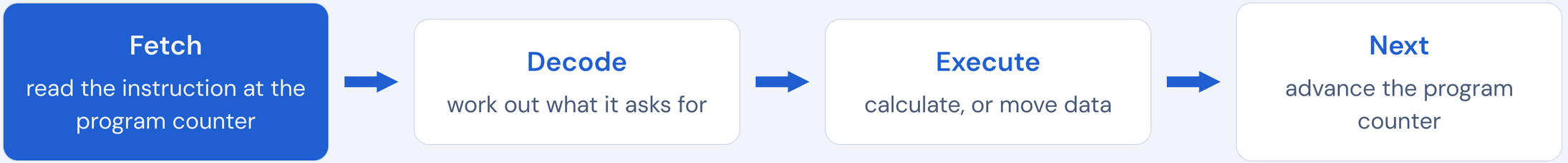
Described by John von Neumann in 1945. Phones, laptops and servers are all built on it.

The parts of a von Neumann machine



The two highlighted boxes and the registers together are the central processing unit (CPU). A bus carries addresses, data and control signals between the parts.

The instruction cycle



Program counter

Holds the address of the next instruction. A jump is an instruction that changes it.

Instruction register

Holds the instruction that is being decoded and executed now.

Data registers

Hold the values in use, for example the two numbers being added and the result.

A processor repeats this cycle billions of times a second, from power-on until power-off.

Three instructions, step by step

Memory before

100: LOAD 200
101: ADD 201
102: STORE 202
…
200: 7
201: 5
202: 0

Program counter	Instruction	What the CPU does	Register
100	LOAD 200	Copies the value in cell 200 into the register	7
101	ADD 201	Adds the value in cell 201 to the register	12
102	STORE 202	Copies the register into cell 202	12

Cell 202 now holds 12. The instructions in cells 100 to 102 and the numbers in cells 200 to 202 are stored in the same way; only the program counter decides which cells are run as instructions.

What a modern computer adds



Caches

Memory is far slower than the processor. Small, fast caches keep recently used instructions and data close to it.



Several cores

Each core is a processor with its own registers, running its own instruction cycle at the same time as the others.



Pipelines

A core works on several instructions at once, each at a different stage of the cycle.



Interrupts

After each instruction the processor checks whether a device needs attention.

The model has not changed: instructions and data are still in one memory, and each core still fetches, decodes and executes.

Why this matters for the operating system

01

A program must be in memory to run

Something has to load it, find room for it, and take the room back afterwards. That is memory management.

02

A core runs one program at a time

Something has to decide which program each core runs, and for how long. That is CPU scheduling.

03

Code and data share one memory

Nothing in the model stops one program from overwriting another. The hardware and the operating system add that protection.

The rest of this part shows how the processor, the memory and the devices are connected, and how they get the processor's attention.

How the parts are connected

CPU
one or more

Disk controller
disks

USB controller
mouse, keyboard, printer

Graphics adapter
monitor

System bus
the common path between all components

Memory
shared by the CPU and the controllers

Device controller

In charge of one type of device, with a local buffer.

Device driver

Operating-system code that knows one controller and gives the rest of the system a uniform interface.

Concurrent work

The CPU and the devices run at the same time. A controller signals that it has finished with an interrupt.

Interrupts

An operating system is *interrupt driven*: it waits for something to happen, then reacts.

Hardware interrupt

A device sends a signal to the CPU, for example when an I/O operation completes.

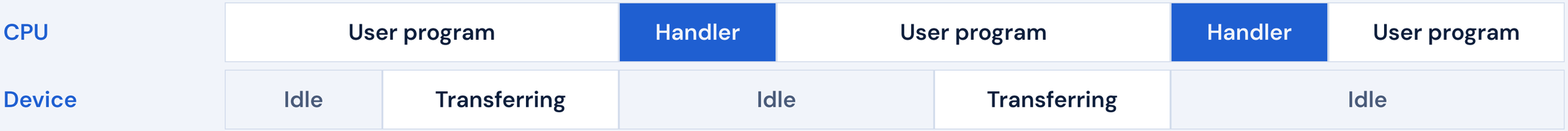
Trap or exception

A software-generated interrupt, caused by an error such as division by zero, or by a request from a user program.

Interrupt vector

A table of addresses, one per kind of interrupt. It tells the CPU where the service routine for each interrupt is.

An interrupt on a timeline



The device works alone

While the device transfers data, the CPU runs the user program. It does not wait.

Done means interrupt

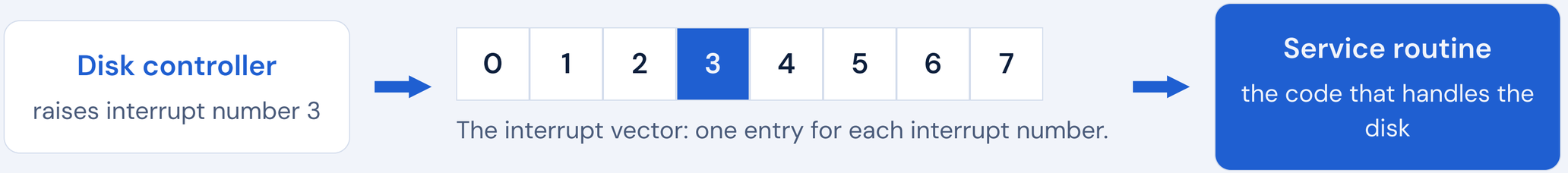
The moment a transfer ends, the device raises an interrupt and the handler starts.

Then back again

The handler is short. When it ends, the user program continues where it stopped.

Time runs from left to right. The two handler blocks begin exactly where the two transfers end.

Finding the right routine



A table of addresses

Each entry holds the address of the routine for one kind of interrupt.

No searching

The interrupt number is the index, so the CPU reaches the routine in one step.

Filled in at start-up

The operating system writes the addresses when it starts.

Handling an interrupt



The interrupted program must not notice anything except the lost time. That is why the state is saved before the routine runs and restored after it.

One read request, step by step

User program
asks to read a file

Operating system
the device driver

Device controller
registers and a local buffer

Device
for example a disk

1. The program asks to read, with a system call.
2. The driver writes the request into the controller's registers.
3. The controller moves the data from the device into its local buffer.
4. The controller raises an interrupt: the transfer is done.
5. The driver copies the data to the program's memory, and the program continues.

The request travels down the four layers, and the data travels back up.

Two ways to wait for I/O

Synchronous: wait for it

Program	Runs	Waits	Runs
Device	Idle	Transfer	Idle

Control returns to the program only when the I/O has completed. One request is outstanding at a time.

Asynchronous: do not wait

Program	Runs	Keeps running	Runs
Device	Idle	Transfer	Idle

Control returns at once. The operating system keeps a device-status table to track every request in progress.

A program asks for I/O with a system call: a request to the operating system to do something on the program's behalf.

Storage structure



Main memory

The only large storage that the CPU can access directly. Random access, and usually volatile: its contents are lost when power is off.



Secondary storage

An extension of main memory that gives large, non-volatile capacity. Hard disk drives and non-volatile memory devices such as solid-state drives.

A program must be loaded into main memory before it can run. Everything else is kept on secondary storage until it is needed.

Storage units

Unit	Size	Note
Bit	0 or 1	The basic unit of computer storage
Byte	8 bits	The smallest convenient chunk on most computers
Word	one or more bytes	The native unit of a given architecture, for example 64 bits
Kilobyte (KB)	1,024 bytes	
Megabyte (MB)	1,024 × 1,024 bytes	1,024 KB
Gigabyte (GB)	1,024 MB	
Terabyte (TB)	1,024 GB	

Manufacturers often round: a megabyte as one million bytes, a gigabyte as one billion bytes.

The storage hierarchy

- **Registers**
- Cache
- Main memory
- Non-volatile memory
- Hard disk drives
- Optical disk
- Magnetic tapes

SPEED

Going up

Faster access, smaller capacity, higher cost per byte. The top levels are volatile.

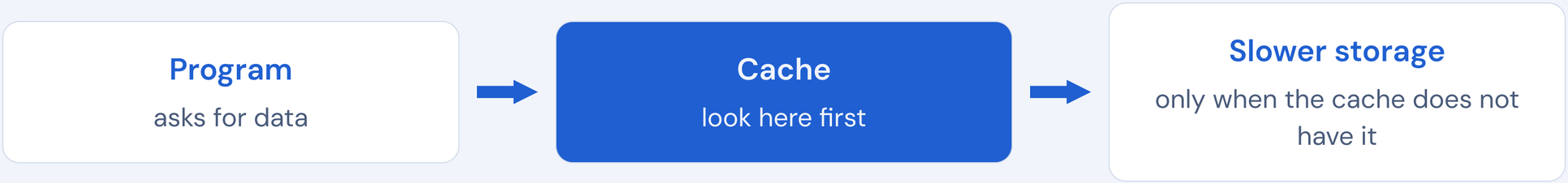
CAPACITY

Going down

Slower access, larger capacity, lower cost per byte. The lower levels keep data without power.

Caching

Copy information that is in use from slower storage to *faster storage*, for a while.



Check the cache first

If the information is there, it is used directly. If not, it is copied into the cache and then used.

A cache is small

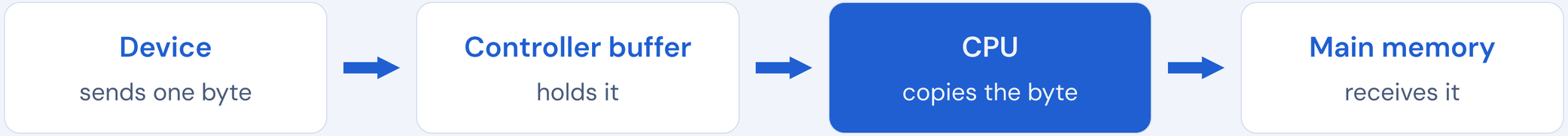
So its size and the choice of what to replace are important design problems.

It happens everywhere

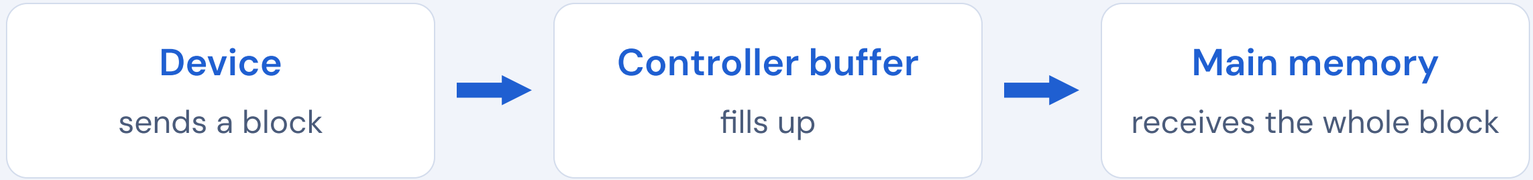
In hardware, in the operating system and in software.

Direct memory access

Without it: the CPU is interrupted for every byte



With it: the controller writes the whole block to memory



With direct memory access there is one interrupt per block, to tell the CPU that the transfer is finished. A fast device would otherwise leave the CPU time for nothing else.

PART 3

Computer-system architecture

From one processor to many, and from one machine to many.

Multiprocessor systems

1

Increased throughput

More work is done in less time, although N processors give less than N times the speed.

2

Economy of scale

Processors share peripherals, storage and power, which costs less than several separate systems.

3

Increased reliability

If one processor fails, the others continue. The system slows down instead of stopping.

Also called parallel systems or tightly coupled systems. Most systems today, from phones to servers, are multiprocessor systems.

Symmetric and asymmetric multiprocessing

ONE IN CHARGE

Asymmetric

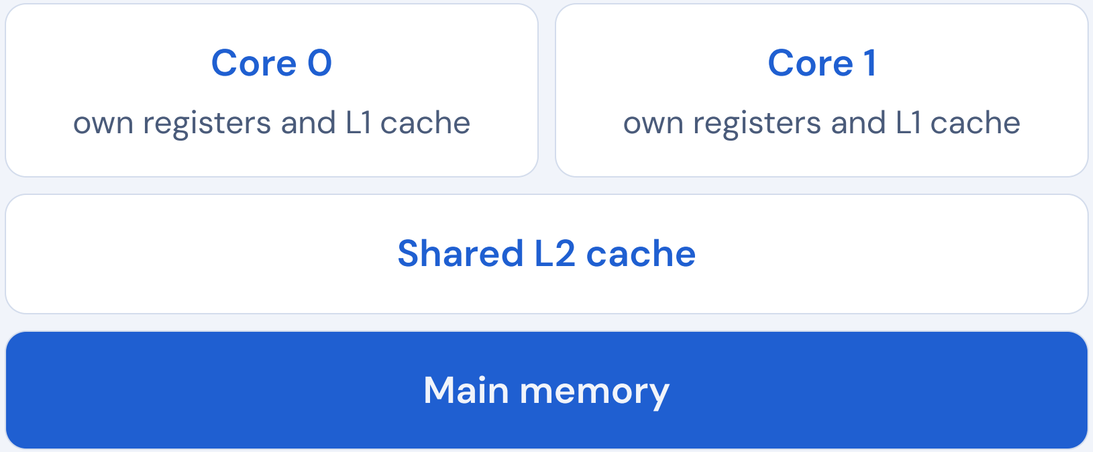
Each processor is given a specific task. One processor controls the system and the others follow its instructions.

ALL EQUAL

Symmetric (SMP)

Each processor performs all tasks, including operating-system functions and user processes. All processors are peers. This is the common design.

Multicore and NUMA



Multicore

Several computing cores on a single chip. Communication on one chip is faster than between chips, and the chip uses less power.

NUMA

Non-uniform memory access: each CPU has its own local memory, reached over a fast local bus. The system scales to more processors, but remote memory is slower.

Clustered systems

Several complete systems working together, usually sharing storage, to give *high availability*.

Asymmetric clustering

One machine is in hot-standby mode. It only monitors the active server, and takes over if that server fails.

Symmetric clustering

Several nodes run applications and monitor each other. All the hardware is used.

High-performance computing

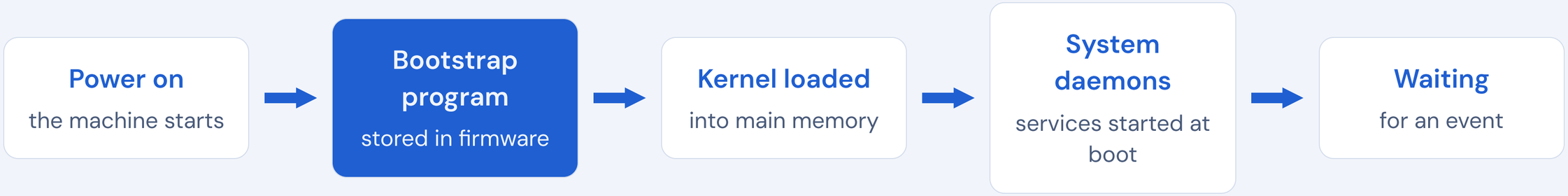
Some clusters exist for speed. The application must be written to run in parallel on the nodes.

PART 4

Operating-system operations

Starting up, keeping many programs running, and staying in control.

Starting the system



The bootstrap program initialises all aspects of the system, then loads the kernel and starts it. After that the kernel waits for interrupts.

Multiprogramming and multitasking

KEEP THE CPU BUSY

Multiprogramming

Several jobs are kept in memory. When the running job has to wait, for I/O for example, the operating system switches to another. The CPU always has something to execute.

KEEP THE USER HAPPY

Multitasking

The logical extension. The CPU switches between jobs so often that users can interact with each program while it runs. Response time should be under a second.

Both need CPU scheduling to choose the next job, and memory management to keep several jobs in memory. Virtual memory lets a job run even when it is not completely in memory.

Multiprogramming on a timeline

CPU	Job A	Job B	Job A	Job B
Job A	Running	Waiting for I/O	Running	Finished
Job B	Ready	Running	Ready	Running

Job A starts I/O

It cannot use the CPU while it waits, so the operating system gives the CPU to Job B.

The I/O completes

An interrupt tells the operating system. Job A is ready again and gets the CPU back.

The CPU is never idle

As long as one job is ready, the CPU has something to execute.

Memory in a multiprogrammed system



The operating system and several processes are in memory at the same time.

Only one of them uses a CPU core at any instant.

When that process waits, the core is given to another process that is ready to run.

A process is a program that has been loaded into memory and is executing.

Dual-mode operation

The hardware tells *user mode* from *kernel mode* with a mode bit.

User mode

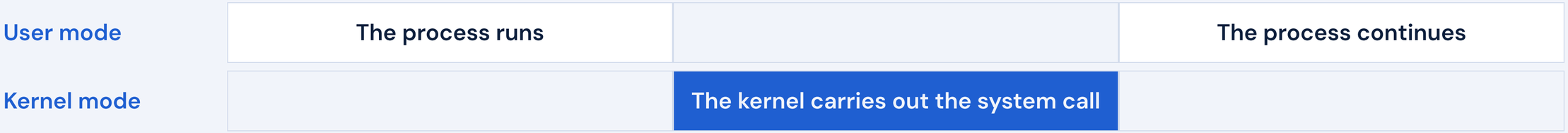
The mode bit is 1. The system is running a user application. Privileged instructions are not allowed and cause a trap.

Kernel mode

The mode bit is 0. The system is running operating-system code. Every instruction is allowed, including control of I/O, the timer and interrupts.

This protects the operating system from user programs, and user programs from each other.

From user mode to kernel mode and back



Going in

The system call causes a trap. The hardware sets the mode bit to 0 and jumps to the kernel.

Coming back

The kernel returns from the system call. The mode bit goes back to 1.

Never by itself

The only way into kernel mode is an interrupt, a trap or a system call. A user program cannot set the mode bit.

The timer

Why

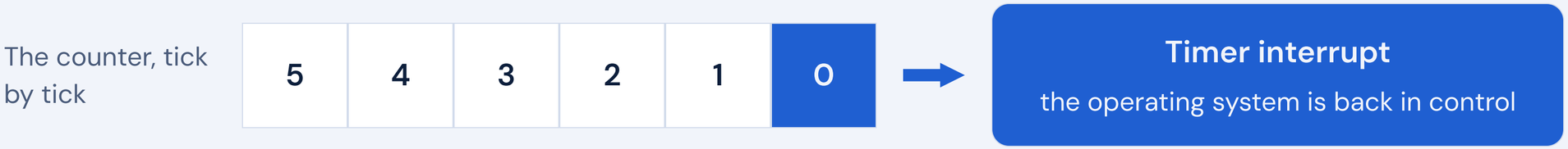
A program could loop forever, or never give the CPU back. The operating system must be sure to regain control.

How

A counter is decreased on every clock tick. When it reaches zero, an interrupt occurs.

Who

Only the operating system sets the timer. The instructions that change it are privileged.



PART 5

Resource management

The operating system is a resource manager: CPU, memory, files, storage and devices.

Process management

A program is *passive*. A process is *active*.

A process is a program in execution. It needs CPU time, memory, files and I/O devices, and gives them back when it ends.

A single-threaded process has one program counter. A multithreaded process has one per thread.

The operating system is responsible for:

- Creating and deleting processes
- Scheduling processes and threads on the CPUs
- Suspending and resuming processes
- Process synchronization
- Process communication

Memory management

To run, a program and its data must be *in main memory*.

Memory management decides what is in memory and when, to make good use of the CPU and respond quickly to users.

The operating system is responsible for:

- Keeping track of which parts of memory are in use, and by which process
- Allocating and deallocating memory space as needed
- Deciding which processes, or parts of them, to move into and out of memory

File-system and mass-storage management

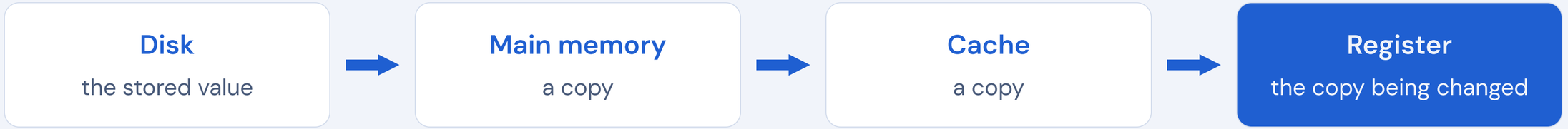
File system

The operating system gives one logical view of stored information: the file. Files are organised into directories, and access control decides who may use them. It creates and deletes files and directories, maps files onto storage, and backs them up.

Mass storage

Disks hold what does not fit in memory and what must be kept. The operating system mounts and unmounts devices, manages free space, allocates storage, schedules disk requests, partitions and protects.

The same data in several places



When a value is updated in a register, the copies below it are out of date. With one process this is simple. With multitasking, several CPUs or several machines, every user must still get the most recent value.

The I/O subsystem

One purpose of an operating system is to *hide* the peculiarities of hardware devices from the user.

Memory management of I/O

Buffering holds data while it is transferred. Caching keeps copies in faster storage. Spooling overlaps the output of one job with the input of others.

A general interface

One device-driver interface, so the rest of the system treats devices alike.

Specific drivers

A driver for each hardware device, which knows that device's details.

PART 6

Protection, virtualization and environments

Who may use what, and where operating systems run today.

Protection and security



Protection

Any mechanism that controls the access of processes or users to the resources the operating system defines.

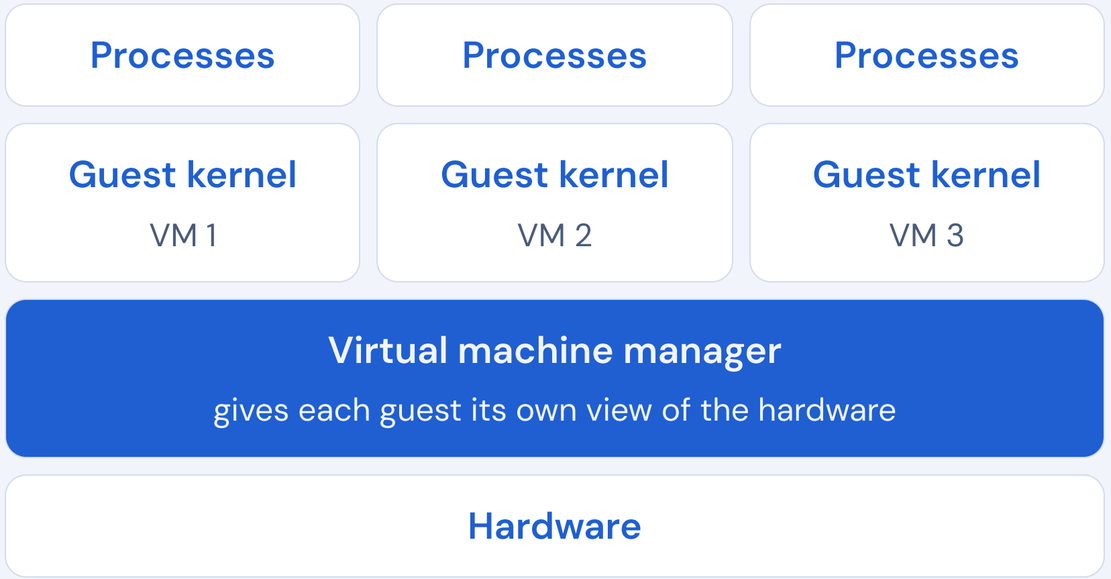


Security

The defence of the system against internal and external attacks: denial of service, worms, viruses, identity theft, theft of service.

Systems first tell users apart. Each user has a user ID, and may belong to groups with a group ID. The IDs are attached to that user's files and processes to decide access. Privilege escalation lets a user change to an ID with more rights.

Virtualization



Virtualization lets an operating system run as an application inside another operating system.

The guest operating system and its applications are compiled for the same CPU as the host, and run natively.

Emulation is different: the source CPU type is not the target CPU type, so every instruction is translated, which is much slower.

Computing environments

Traditional

Stand-alone general-purpose machines, now almost always connected to a network.

Mobile

Smartphones and tablets, with extra features such as GPS and motion sensors.

Client–server

Servers respond to the requests that clients generate: files, databases, computation.

Peer-to-peer

No distinction between clients and servers. Every node may act as either.

Cloud computing

Computing, storage and applications delivered as a service across a network.

Real-time embedded

Special-purpose systems with fixed time limits. The most common kind of computer.

Free and open-source operating systems



Open source

The source code is available, not only the compiled binary. Anyone can read it, learn from it and change it.



Examples

GNU/Linux, the BSD UNIX family including the Darwin core of macOS, and many more.



Why it matters to us

We can study a real kernel, modify it, compile it and run it in a virtual machine without risk to our own computer.

Free software goes further than open source: its licence also guarantees the freedom to use, change and redistribute it.

Key takeaways

01

An intermediary

The operating system sits between programs and hardware, and manages resources for both.

02

Interrupt driven

Devices, errors and system calls all reach the kernel as interrupts.

03

Two modes

User mode and kernel mode, plus the timer, keep the operating system in control.

04

A resource manager

Processes, memory, files, storage and I/O are each managed for all programs at once.

Next session

COMING UP

Operating-system services

The services an operating system provides, system calls, and how operating systems are structured.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 1.